

DEVADN

Évaluation OWASP ASVS

devadn.com — août 2026

De l'installation à l'utilisation

2 août 2026

Sommaire

1. Évaluation OWASP ASVS	3
Synthèse	3
V1 — Architecture, conception	3
V2 — Authentification	3
V3 — Gestion de session	4
V4 — Contrôle d'accès	4
V5 — Validation, encodage	4
V6 — Cryptographie stockée	5
V7 — Gestion des erreurs et journalisation	5
V8 — Protection des données	5
V9 — Communications	5
V11 — Logique métier	5
V12 — Fichiers et ressources	5
V13 — API et handlers	6
V14 — Configuration	6
Conclusion	6

CHAPITRE 1

Évaluation OWASP ASVS

Vérification du code contre l'OWASP Application Security Verification Standard (v4.0.3), niveau L1 (socle) et exigences L2 pertinentes. Chaque ligne cite une preuve dans le code ou une vérification live sur <https://devadn.com>.

Pile évaluée : Next.js 16 (App Router, Server Actions), Auth.js v5 (JWT), Prisma 7 + PostgreSQL, déployée en conteneur Docker derrière Caddy sur un VPS.

Synthèse

| Chapitre | Verdict | |---|---| | V1 Architecture | Conforme | | V2 Authentification | **1 écart corrigé** (tokens de reset en clair) | | V3 Gestion de session | Conforme | | V4 Contrôle d'accès | Conforme (modèle à 3 axes, appliqué en base) | | V5 Validation / encodage | Conforme | | V6 Cryptographie stockée | Conforme après correctif V2 | | V7 Erreurs & journalisation | Conforme | | V8 Protection des données | Conforme | | V9 Communications | Conforme | | V11 Logique métier | Conforme | | V12 Fichiers | Conforme | | V13 API / handlers | Conforme | | V14 Configuration | Conforme (app) — 2 points serveur à confirmer |

Un seul écart applicatif a été trouvé, et il est corrigé : les tokens de réinitialisation de mot de passe étaient stockés en clair (V2.5.4). Ils sont désormais hachés en SHA-256. Le reste de l'application satisfait le socle L1.

V1 — Architecture, conception

- **V1.1 / V1.4** — Les deux axes d'accès (rôle via matrice de capacités, fonction via sections + portée chantier) sont résolus **en un seul endroit**, [lib/accessContext.ts](#), appelé par chaque surface. Contrôle centralisé, pas éparpillé. **PASS**
- **V1.2.3** — L'authentification passe par un composant unique (Auth.js + [lib/authorizeCredentials.ts](#)), pas de logique de mot de passe dupliquée. **PASS**

V2 — Authentification

- **V2.1.1 / V2.1.7** — Mots de passe hachés avec **bcrypt** (coût 10), [service/auth.ts](#), [actions/users/users.ts](#). Jamais stockés ni journalisés en clair. **PASS**
- **V2.2.1 (anti-automation)** — Limiteur persistant en base : login **5 tentatives / 15 min par email, **20 par IP** ([lib/loginRateLimit.ts](#)) ; **demande de reset** 3 / 15 min par email, **10 par IP** ([actions/auth/auth.ts](#)). **PASS****
- **V2.2.x (énumération de comptes)** — La comparaison de mot de passe tourne contre un **DUMMY_HASH** même quand l'email n'existe pas, égalisant le temps de réponse ([service/auth.ts](#)). Le flux « mot de passe oublié » renvoie toujours la même réponse générique, que le compte existe ou non — et une panne du fournisseur d'email est capturée pour ne pas trahir l'existence par une erreur ([actions/auth/auth.ts](#)). **PASS**
- **V2.5.1 / V2.5.4 (tokens de récupération)** — *Écart initial : le token de reset était stocké en clair.* **CORRIGÉ** : seul **SHA-256(token)** est stocké ([lib/resetToken.ts](#), [repository/users.ts](#)), le token brut n'existe que dans l'URL du mail. Token à usage unique ([usedAt](#)), expirant, 32 octets d'entropie ([randomBytes\(32\)](#)). **PASS (après correctif)**

- **V2.5.3** — Le lien de reset ne fait jamais confiance à l'en-tête `Host` en

production : `AUTH_URL` est requis, sinon l'envoi est abandonné — pas d'empoisonnement du lien (`getResetBaseUrl`). **PASS**

V3 — Gestion de session

- **V3.2.1 / V3.4** — Sessions JWT gérées par Auth.js. Cookies vérifiés en

production : `__Host-authjs.csrf-token` et `__Secure-authjs.callback-url` avec **HttpOnly ; Secure ; SameSite=Lax**. Le préfixe `__Host-` interdit l'écrasement de cookie inter-sous-domaine. **PASS**

- **V3.3.1** — Déconnexion via `signOut` d'Auth.js, invalide la session.

PASS

- **V3.5 (callbacks de rôle)** — Le rôle est porté dans le JWT via des callbacks

placés dans la config **partagée** (`auth.config.ts`), pour que le proxy edge le voie — sans quoi la frontière du portail CLIENT ne se déclencherait pas. **PASS**

V4 — Contrôle d'accès

- **V4.1.1 / V4.1.3 (appliqué côté serveur)** — Trois couches, ****toutes**

serveur : ``requireCapability`` (rôle), ``requireSectionAccess`` (fonction !' section), ``requireProjectAccess`` (fonction !' chantier), plus ``requireAppUser`` pour les route handlers. Un test structurel (`tests/authz-coverage.test.ts`) échoue si une action exportée d'une section n'appelle pas sa garde. **PASS****

- **V4.1.2 (pas de contrôle par masquage)** — Le filtrage par portée se fait

dans la requête, pas au rendu : listes projets/entreprises, page projet, tableau de bord, exports CSV, rapport PDF. Un chantier hors portée est ***introuvable***, y compris par URL directe (`lib/accessContext.ts`, `repository/projects.ts`, `repository/clients.ts`). **PASS**

- **V4.2.1 (IDOR)** — Les mutations relisent le `projectId` réel depuis la base

et vérifient l'appartenance avant d'agir ; les suppressions de dossiers/plans sont scopées au projet (`deleteMany/updateMany` filtrés). Un id égaré ne peut pas toucher les données d'un autre projet. **PASS**

- **V4.3.1 (frontière portail)** — Un login CLIENT est confiné à `/portail` par

le proxy et par `blockClientFromApp()` dans le rendu (défense en profondeur) ; sa portée projet vient de `getPortalContext`. Un CLIENT est aussi refusé sur les route handlers d'app. **PASS**

- **V4.3.2** — L'énumération de répertoire est neutralisée : ``object-src``

`'none'`, pas de listing, et un projet hors périmètre répond « introuvable » plutôt que « interdit », pour ne pas confirmer son existence. **PASS**

V5 — Validation, encodage

- **V5.1.1 (validation d'entrée)** — Toute entrée de mutation passe par un

schéma **Zod** (`schemas/*.ts`) avant traitement. **PASS**

- **V5.2.x (encodage de sortie)** — React échappe par défaut ; le générateur

d'email échappe explicitement toute donnée interpolée et rejette les URL non http(s) (`lib/email/render.ts`, testé). **PASS**

- **V5.3.4 (injection SQL)** — Accès données exclusivement via Prisma

(paramétré). Aucune requête SQL concaténée. **PASS**

- **V5.1.3 (redirections)** — Pas de redirection ouverte : le lien de reset est construit depuis `AUTH_URL` fixé, jamais depuis une entrée utilisateur. **PASS**

V6 — Cryptographie stockée

- **V6.2.1** — Mots de passe : bcrypt. Tokens de reset : SHA-256 (après le correctif V2). Secret de session : `AUTH_SECRET` hors dépôt, dans le `.env` serveur. **PASS**
- **V6.2.2** — Entropie des tokens via `crypto.randomBytes`, pas de générateur faible. **PASS**

V7 — Gestion des erreurs et journalisation

- **V7.1.1 (pas de fuite dans les logs)** — Aucun mot de passe, token ou secret journalisé. Les logs d'erreur repository sont génériques (`getErrorMessage` renvoie un message de repli, pas de stack exposée à l'utilisateur). **PASS**
- **V7.4.1 (messages génériques)** — Les réponses d'erreur utilisateur sont génériques et localisées ; les détails restent côté serveur. **PASS**

V8 — Protection des données

- **V8.2.2 (données sensibles hors client)** — `findAll` (utilisateurs) sélectionne explicitement sans le hash de mot de passe. Les exports CSV sont gardés par `requireAppUser`. **PASS**
- **V8.3.1** — `Cache-Control: no-store` sur les réponses PDF sensibles. **PASS**

V9 — Communications

- **V9.1.1 (TLS)** — Servi en HTTPS derrière Caddy. **HSTS** vérifié live : `max-age=31536000 ; includeSubDomains`. **PASS**
- **V9.2.1** — L'app écoute sur `127.0.0.1:3000` seulement ; le reverse proxy gère le TLS. Le fetch d'images du générateur PDF est restreint à `res.cloudinary.com` en HTTPS (anti-SSRF). **PASS**

V11 — Logique métier

- **V11.1.1 (séquençage)** — Le scan de bulletin s'applique en **une transaction ; l'ajout de matériel cumule dans une transaction (anti-course). Les numéros de réserve viennent d'un compteur monotone par projet, jamais réutilisés. **PASS****
- **V11.1.4 (anti-automation)** — Limiteurs sur login et reset (cf. V2.2.1). **PASS**

V12 — Fichiers et ressources

- **V12.1.1 / V12.2.1 (type de fichier)** — Upload vérifié par MIME déclaré et extension ; SVG et types dangereux explicitement bloqués (`lib/cloudinary.ts`, `BLOCKED_UPLOAD_MIME_TYPES`). **PASS**
- **V12.3.1 (chemin)** — Le nom de fichier du rapport PDF est slugifié en ASCII, neutralisant l'injection d'en-tête `Content-Disposition` (testé). **PASS**

- **V12.6.1 (SSRF via récupération)** — Allowlist d'hôte + timeout + plafond de taille sur le fetch d'images serveur. **PASS**

V13 — API et handlers

- **V13.1.1 (authz sur les handlers)** — Le proxy protège `/api/**` par défaut ; seuls `/api/auth` et `/api/health` sont publics par exception explicite (`lib/routeGuard.ts`). Les handlers d'export vérifient en plus `requireAppUser` en interne (double couche). **PASS**
- **V13.2.1 (CSRF)** — Aucun route handler mutant : toutes les mutations passent par des Server Actions, protégées nativement par la vérification d'origine de Next.js. Les exports sont en GET, sans effet de bord. **PASS**

V14 — Configuration

- **V14.3.2 (pas de divulgation)** — `poweredByHeader: false`, en-tête `Server` retiré au niveau de Caddy — vérifié absent en prod. **PASS**
- **V14.4.x (en-têtes de sécurité)** — Vérifiés live : CSP à **nonce** sans `unsafe-inline`, `object-src 'none'`, `frame-ancestors 'none'`, `base-uri 'self'`, `form-action 'self'` ; `X-Content-Type-Options: nosniff` ; `X-Frame-Options: DENY` ; `Referrer-Policy` ; `Permissions-Policy` ; COOP/CORP. **PASS**
- **V14.1.x (dépendances)** — `npm audit` : 0 ; Snyk : 0 chemin vulnérable ; Actions GitHub épinglées sur SHA. **PASS**

Points serveur (hors code — à confirmer sur le VPS)

Ces éléments relèvent de l'exploitation du serveur, pas du code, et sont documentés dans [docs/SECURITE-SSH.md](#) :

1. **fail2ban** lisait `/var/log/auth.log` (inexistant sur Ubuntu 24.04) au lieu du backend `systemd` — à basculer, sinon la prison ne bannit rien.
2. **Pare-feu `ufw`** à activer (n'exposer que 80/443/SSH).

L'authentification SSH par mot de passe a été désactivée (clé uniquement) et le durcissement est en place.

Conclusion

L'application satisfait le socle **ASVS L1** et l'essentiel des exigences L2 pertinentes. Le seul écart applicatif — le stockage en clair des tokens de réinitialisation (V2.5.4) — a été **trouvé et corrigé** au cours de cette évaluation, avec tests à l'appui. Les deux points restants sont des réglages d'exploitation serveur, sans rapport avec le code.